

ObservaAção

Ouvidoria Cidadã Digital

Relatório de Métricas — SonarCloud

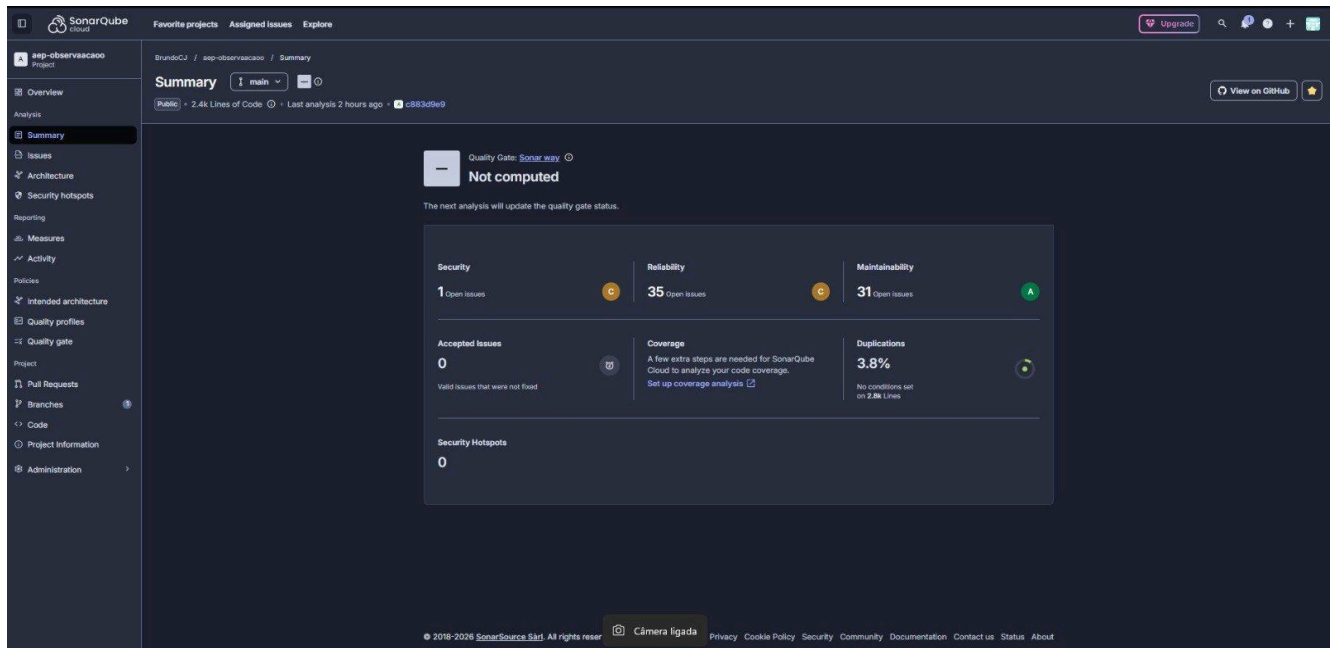
AEP 2º Bimestre

Bruno Conrado Jeanini · Victor Hugo Pacchioni Sgobbi · Leonardo Camilotti Moreno

github.com/Leocm123/aep-observacao

1. Overview — Painel de Qualidade

O SonarCloud analisou o projeto ObservaAção no branch main



Resumo das métricas coletadas:

- Linhas de código analisadas: 2.4k (2.8k incluindo comentários e espaços em branco)
- Security: 1 issue aberto — Grau C (vulnerabilidade Major)
- Reliability: 35 issues abertos — Grau C
- Maintainability: 31 issues abertos — Grau A
- Duplicações: 3.8% do código
- Coverage: não configurada nesta análise (requer setup adicional do JaCoCo)
- Security Hotspots: 0 identificados

2. Maintainability — Manutenibilidade

Avalia a facilidade de compreender, modificar e evoluir o código ao longo do tempo. O SonarCloud identificou 31 issues. O projeto recebeu Grau A (o melhor possível), indicando que o débito técnico acumulado é baixo em relação ao tamanho do código.

Caso 1 — LocalDateTime.now() sem ZoneId especificado

Code Smell · Severidade: Info · Esforço: 5 min por ocorrência

Arquivos afetados:

Movimentacao.java, Solicitacao.java, SolicitacaoService.java, GeradorProtocolo.java

The screenshot shows the SonarCloud interface for the project 'semp-observacao'. The left sidebar contains navigation links for Overview, Summary, Issues, Architecture, Security hotspots, Reporting, Measures, Activity, Policies, Intended architecture, Quality profiles, Quality gate, Project, Pull Requests, Branches, Code, Project Information, and Administration. The main panel displays the 'Issues' section for the file 'src/main/java/br/com/observacao/Movimentacao.java'. It shows a list of 2/31 issues, with the first issue highlighted: 'Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method.' The issue is categorized as 'Intentionality / Not logical' and 'Time-based \".now()\" methods should specify a ZoneId or a Clock [java:S5868]'. The code snippet shows a constructor for the 'Solicitacao' class with various fields and a call to 'LocalDateTime.now()' without a 'ZoneId'.

The screenshot shows the SonarCloud interface for the project 'semp-observacao'. The left sidebar contains navigation links for Overview, Summary, Issues, Architecture, Security hotspots, Reporting, Measures, Activity, Policies, Intended architecture, Quality profiles, Quality gate, Project, Pull Requests, Branches, Code, Project Information, and Administration. The main panel displays the 'Issues' section for the file 'src/main/java/br/com/observacao/Movimentacao.java'. It shows a list of 2/31 issues, with the first issue highlighted: 'Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method.' The issue is categorized as 'Intentionality / Not logical' and 'Time-based \".now()\" methods should specify a ZoneId or a Clock [java:S5868]'. The code snippet shows a constructor for the 'Solicitacao' class with various fields and a call to 'LocalDateTime.now()' without a 'ZoneId'. Below the code snippet, there is a 'Noncompliant code example' and a 'Compliant solution'.

Noncompliant code example

```
LocalDateTime dt = LocalDateTime.now(); // Noncompliant
```

Compliant solution

```
// Option 1: Make system default explicit
LocalDateTime dt1 = LocalDateTime.now(ZoneId.systemDefault());

// Option 2: Use a specific time zone
LocalDateTime dt2 = LocalDateTime.now(ZoneId.of("Europe/Lisbon"));

// Option 3: Use UTC for server-side operations
LocalDateTime dt3 = LocalDateTime.now(ZoneId.of("UTC"));

// Option 4: Provide a Clock
Clock myClock = Clock.systemUTC();
LocalDateTime dt4 = LocalDateTime.now(myClock);
```

Como isso impacta:

- O método `LocalDateTime.now()` usa implicitamente o fuso horário padrão da JVM, que pode variar entre ambientes de desenvolvimento e produção.
- Em servidores cloud configurados em UTC, a data registrada ficaria 3 horas adiantada em relação ao horário de Brasília.
- Pode gerar inconsistências no cálculo do prazo SLA e no histórico de movimentações.
- A intenção do código não é explícita, dificultando a manutenção por outros desenvolvedores.

Como resolver:

- Substituir por `LocalDateTime.now(Zoneld.of("America/Sao_Paulo"))` em todos os pontos.
- Criar uma constante `ZONE_BR = Zoneld.of("America/Sao_Paulo")` reutilizável em todo o projeto.
- Para ambientes em UTC: usar `LocalDateTime.now(Zoneld.of("UTC"))` e converter na camada de apresentação.

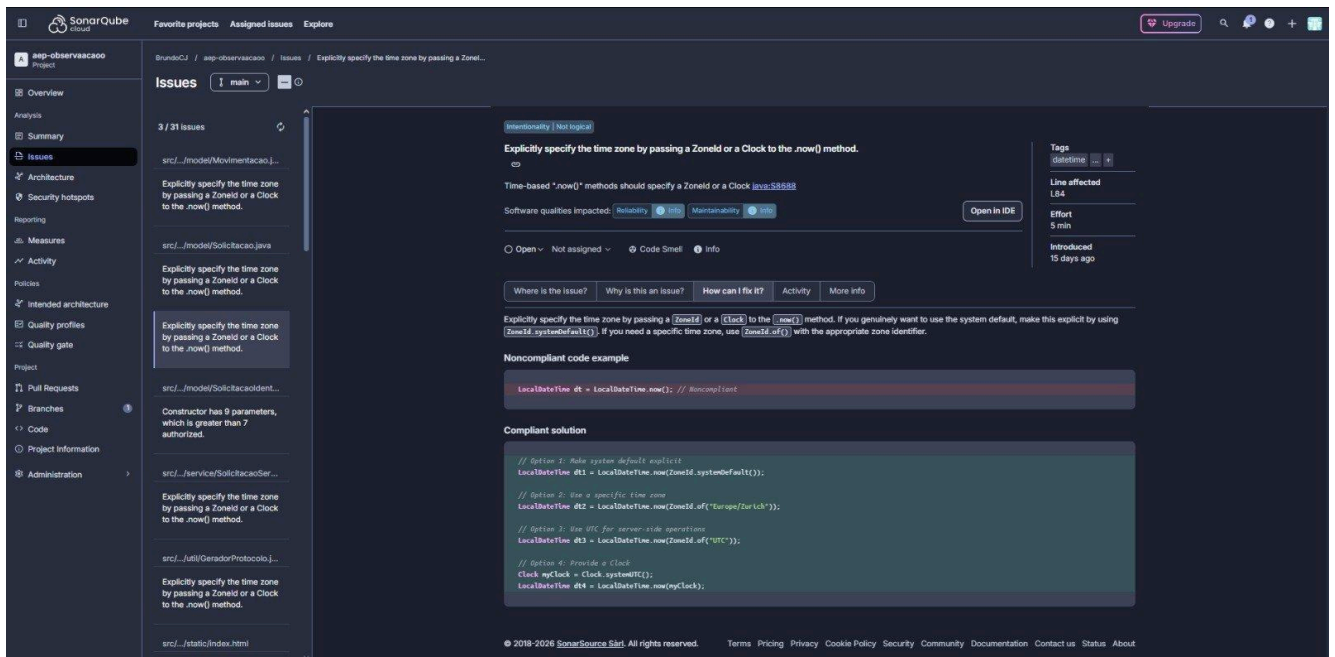
Caso 2 — Construtor com mais de 7 parâmetros

Code Smell · Severidade: Info · Esforço: 30 min

Arquivo afetado:

`Solicitacaoidentificada.java` — construtor com 9 parâmetros (limite recomendado: 7)

The screenshot shows the SonarQube web interface. On the left, there's a sidebar with navigation links. The main area displays an issue titled "Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method." This is a Code Smell issue with a severity of "Info" and an estimated effort of 30 minutes. The issue is located in the file `src/_/model/SolicitacaoIdentificada.java` at line 84. The code snippet shows a constructor with 9 parameters, and the issue highlights the use of `LocalDateTime.now()` without a specified time zone. The interface includes a sidebar with navigation options like Overview, Analysis, Summary, Issues, Architecture, Security hotspots, Reporting, Measures, Activity, Policies, Intended architecture, Quality profiles, Quality gate, Project, Pull Requests, Branches, Code, Project Information, and Administration. The main panel displays the issue details, including the code snippet and the suggested fix.



Como isso impacta:

- Um construtor com 9 argumentos é difícil de ler, chamar corretamente e testar.
- A ordem dos parâmetros pode ser trocada acidentalmente sem que o compilador detecte o erro.
- Qualquer novo campo exige alteração do construtor e de todos os locais que o chamam.
- Indica que a classe pode estar acumulando responsabilidades demais — possível violação do SRP.

Como resolver:

- Aplicar o padrão Builder: criar `SolicitacaoIdenticada.Builder` com métodos encadeados.
- Criar um Value Object `DadosPessoais`(nome, email, telefone) para agrupar os campos relacionados.
- O construtor ficaria: `SolicitacaoIdenticada(protocolo, dadosPessoais, dadosOcorrencia)`.
- Os DTOs de request já aplicam parcialmente essa separação — o próximo passo é refletir no domínio.

3. Reliability — Confiabilidade

Analisa a probabilidade de o sistema se comportar incorretamente em tempo de execução. Com 35 issues e Grau C, esta é a área que merece maior atenção. A maioria dos issues está relacionada ao uso de datas e horários sem considerar fuso horário, o que pode gerar resultados incorretos dependendo do ambiente de execução.

Caso 1 — `LocalDateTime.now()` afetando a confiabilidade dos dados

Code Smell · Impacto: Reliability + Maintainability · Esforço: 5 min por ocorrência

Arquivos afetados:

`Solicitacao.java` (L62), `SolicitacaoService.java` — o mesmo problema de timezone afeta diretamente a confiabilidade dos dados persistidos.

The screenshot shows the SonarQube interface for a project named 'seu-observacao'. The 'Issues' tab is active, displaying a list of 2/35 issues. The selected issue is a 'Code Smell' titled 'Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method.' located in the file 'src/main/java/Solicitacao.java' at line 62. The issue description states: 'Time-based ".now()" methods should specify a ZoneId or a Clock [java:58588]'. The software qualities impacted are 'Reliability' and 'Maintainability'. The effort to resolve the issue is estimated at 5 minutes. The issue was introduced 15 days ago. The code snippet shows the following lines:

```
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78
```

The code is a Java class named 'Solicitacao' with a constructor and several methods. The issue is highlighted on line 62, which is the call to 'LocalDateTime.now()'.

The screenshot shows the same SonarQube interface, but with a detailed view of the issue. The issue description is expanded, showing the rule 'Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method.' and the link to the Java documentation [java:58588]. The 'Noncompliant code example' is shown as:

```
LocalDateTime dt = LocalDateTime.now(); // Noncompliant
```

The 'Compliant solution' is provided as follows:

```
// Option 1: Make system default explicit  
LocalDateTime dt1 = LocalDateTime.now(ZoneId.systemDefault());  
  
// Option 2: Use a specific time zone  
LocalDateTime dt2 = LocalDateTime.now(ZoneId.of("Europe/Zurich"));  
  
// Option 3: Use UTC for server-side operations  
LocalDateTime dt3 = LocalDateTime.now(ZoneId.of("UTC"));  
  
// Option 4: Provide a Clock  
Clock myClock = Clock.systemUTC();  
LocalDateTime dt4 = LocalDateTime.now(myClock);
```

The footer of the page indicates the copyright is © 2019-2026 SonarSource SA. All rights reserved. and provides links to Terms, Pricing, Privacy, Cookie Policy, Security, Community, Documentation, Contact us, Status, and About.

Como isso impacta:

- O dataAbertura registrado pode ser inconsistente entre ambientes: desenvolvimento (horário local) vs. produção (UTC em cloud).
- O cálculo do prazo SLA (`dataAbertura.plusDays(slaDias)`) produz datas corretas somente se o timezone for consistente.
- Um prazo calculado incorretamente pode fazer um atendente acreditar estar no prazo quando já está vencido.
- Em servidores de nuvem (AWS, GCP, Azure), que geralmente operam em UTC, o impacto é amplificado.

Como resolver:

- Padronizar o uso de `ZonedDateTime` ou `OffsetDateTime` em todo o domínio.
- Forma mais simples: `LocalDateTime.now(Zoneld.of("America/Sao_Paulo"))` em todos os pontos.
- Configurar `spring.jackson.time-zone=America/Sao_Paulo` no `application.properties`.
- Correção de baixo esforço e alto impacto na confiabilidade do sistema.

Caso 2 — Cálculo de duração sem tipos conscientes de timezone

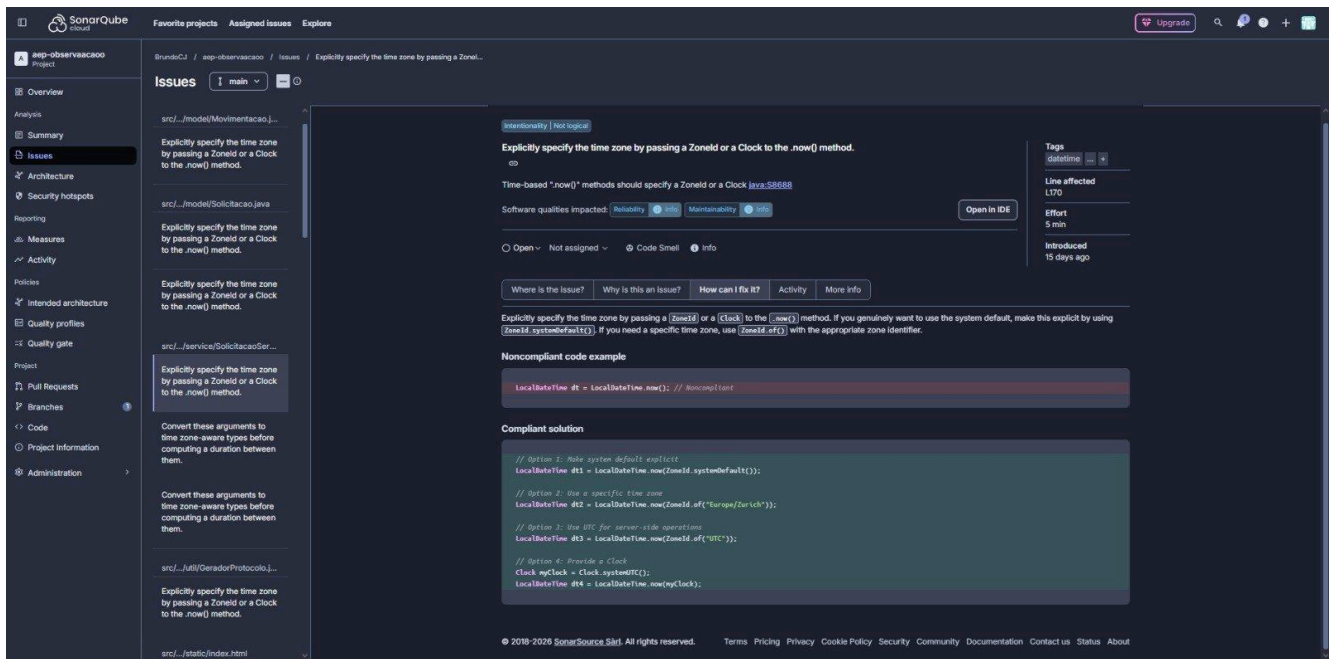
Code Smell · Impacto: Reliability · Esforço: 15 min

Arquivo afetado:

`SolicitacaoService.java` — método `calcularIndicadorSla()` — uso de `ChronoUnit.MINUTES.between()` com `LocalDateTime` sem `Zoneld`.

The screenshot shows the SonarQube web interface. On the left, the 'Issues' sidebar is active, displaying a list of issues. The main panel shows a detailed view of a 'Code Smell' issue titled 'Explicitly specify the time zone by passing a Zoneld or a Clock to the .now() method.' The issue is located in the file `src/_/model/Solicitacao.java` at line 179. The code snippet shows the use of `LocalDateTime.now()` without specifying a time zone. The issue description explains that time-based `.now()` methods should specify a `Zoneld` or a `Clock`. The right sidebar shows tags like 'QualityTime', 'Line affected LTO', 'Effort 5 min', and 'Introduced 15 days ago'. Below the issue description, there are tabs for 'Where is the issue?', 'Why is this an issue?', 'How can I fix it?', 'Activity', and 'More info'. The 'Where is the issue?' tab is selected, showing the code context around line 179.

```
166     };
167   }
168
169   private String calcularIndicadorSla(Solicitacao solicitacao) {
170     LocalDateTime agora = LocalDateTime.now();
171
172     LocalDateTime prazo = solicitacao.calcularPrazo();
173     LocalDateTime abertura = solicitacao.getAbertura();
174
175     if (agora.isAfter(prazo)) {
176       return "VENCIDO";
177     }
178
179     long totalMinutos = ChronoUnit.MINUTES.between(abertura, prazo);
180     long minutosRestantes = ChronoUnit.MINUTES.between(agora, prazo);
181
182     if (totalMinutos == 0) {
183       return "VENCIDO";
184     }
185
186     double percentualRestante = (double) minutosRestantes / totalMinutos;
187
188     if (percentualRestante > 0.5) {
189       return "VERDE";
190     } else if (percentualRestante > 0.25) {
191       return "AMARELO";
192     } else {
193       return "VENCIDO";
194     }
195   }
196 }
```



Como isso impacta:

- ChronoUnit.MINUTES.between(abertura, prazo) só funciona corretamente se ambos os argumentos estiverem no mesmo timezone.
- Durante transições de horário de verão (DST), um intervalo de 60 minutos pode ser calculado como 59 ou 61 minutos.
- O indicador SLA (VERDE / AMARELO / VENCIDO) pode ser exibido incorretamente, afetando decisões de priorização do atendente.
- O percentual restante do prazo fica impreciso, podendo alterar o indicador entre estados sem refletir a realidade.

Como resolver:

- Substituir LocalDateTime por ZonedDateTime.now(ZoneId.of("America/Sao_Paulo")).
- Usar ChronoUnit.MINUTES.between(abertura.toInstant(), prazo.toInstant()) para cálculo baseado em UTC.
- Ou comparar via epochSecond: abertura.toEpochSecond() vs prazo.toEpochSecond().
- Garante que o indicador SLA seja correto independente de horário de verão ou configuração do servidor.

4. Security — Segurança

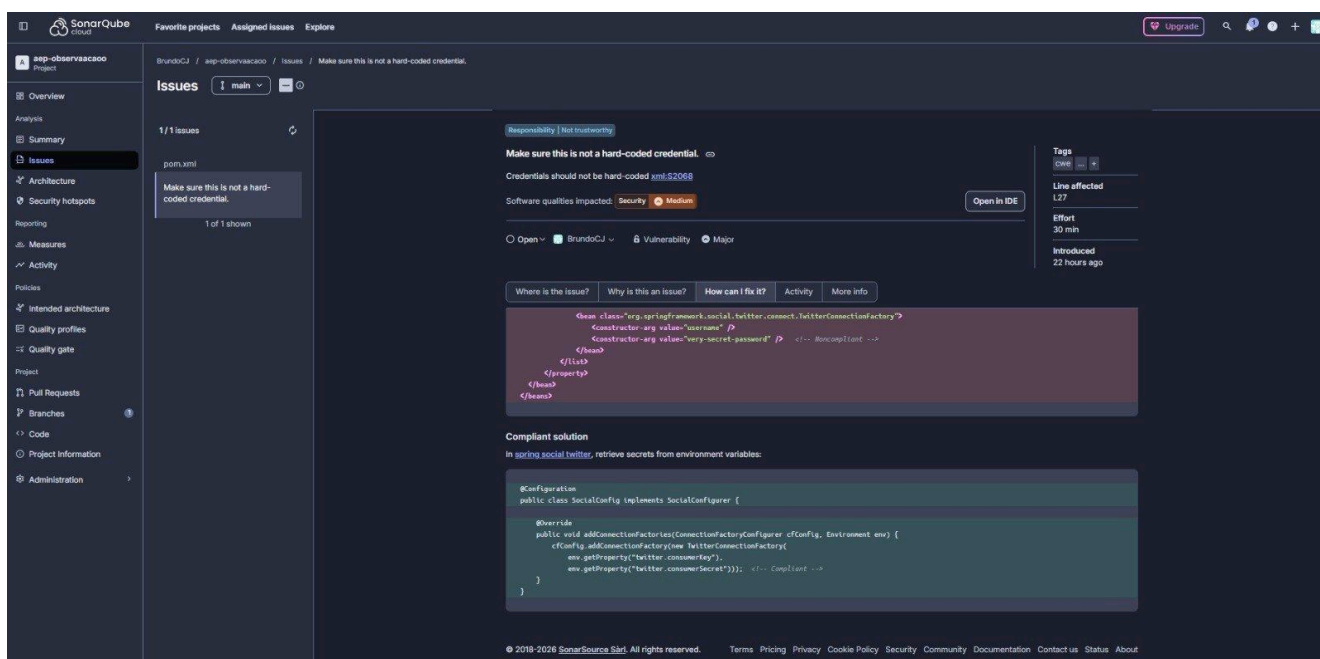
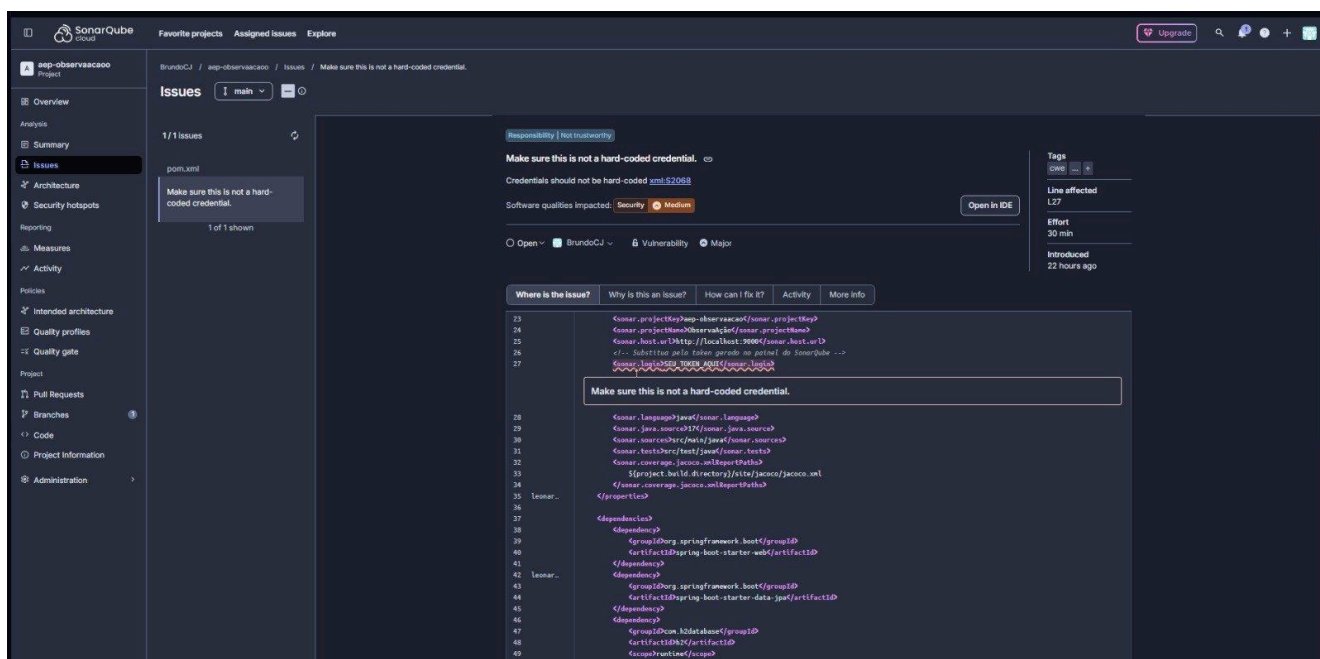
Identifica vulnerabilidades que podem comprometer a integridade, confidencialidade ou disponibilidade do sistema. Com 1 issue aberto de severidade Major e Grau C, o projeto possui um ponto crítico que deve ser corrigido antes de qualquer exposição pública do código ou execução em produção.

Caso 1 — Credencial hard-coded no pom.xml

Vulnerabilidade · Severidade: Major · CWE-798 · Esforço: 30 min

Arquivo afetado:

pom.xml — linha 27 — propriedade <sonar.login>SEU_TOKEN_AQUI</sonar.login>



Como isso impacta:

- Qualquer token real inserido neste campo estaria exposto publicamente, pois o repositório é público no GitHub.
- CWE-798 (Hard-coded Credentials) é uma das vulnerabilidades mais exploradas — qualquer pessoa com acesso ao repo poderia usar o token.
- Um token comprometido permite acesso total ao projeto no SonarCloud, podendo alterar quality gates e suprimir issues.
- Estabelece um padrão ruim: outros desenvolvedores poderiam inserir credenciais reais por engano seguindo o mesmo template.

Como resolver:

- Remover completamente a propriedade `<sonar.login>` do `pom.xml`.
- Passar o token apenas no momento da análise: `.\mvnw.cmd sonar:sonar -Dsonar.login=TOKEN`.
- Para CI/CD com GitHub Actions: usar secrets — `secrets.SONAR_TOKEN` — nunca commitar o valor.
- Adicionar `sonar.properties` ao `.gitignore` para impedir que arquivos de configuração local sejam versionados.

Considerações Finais

A análise do SonarCloud evidencia que o projeto possui uma base de código bem organizada: Grau A em Maintainability com apenas 3.8% de duplicação. Os pontos de melhoria concentram-se principalmente no tratamento de datas sem timezone explícito, issue de maior recorrência presente em múltiplos arquivos e categorias.

A vulnerabilidade de segurança identificada é de correção simples e deve ser resolvida antes da próxima análise para que o Quality Gate compute corretamente. Com as correções aplicadas, o projeto tem condições de atingir Grau B ou superior nas três categorias.

Ações prioritárias identificadas:

- CRÍTICO — Remover `sonar.login` do `pom.xml` (CWE-798)
- ALTA — Substituir `LocalDateTime.now()` por `LocalDateTime.now(Zoneld...)` em todos os arquivos
- MÉDIA — Corrigir `ChronoUnit.MINUTES.between()` para usar tipos conscientes de timezone
- BAIXA — Refatorar `SolicitacaoIdentificada` para reduzir parâmetros do construtor
- FUTURO — Configurar coverage com JaCoCo e criar testes unitários (`src/test` ainda ausente)